

Rapport — Réseau de vols, back-end MongoDB

Titre du projet : Réseau de vols — back-end MongoDB **Membres :** Mathieu Ponnou · Navid Sabete **IPSSI Montpellier, Mastère Dev Data & IA 4^e année · module MIA4** **Date :** 2026-08-28 **Date limite de remise :** 28/08/2026, 23h59 **URL du dépôt Git :** <https://github.com/navidsabete/transport-aerien-ipssi-nsi-mp>

i) Problématique

Contexte métier

Une compagnie aérienne, un comparateur de vols ou un service d'analyse du trafic aérien a besoin de connaître la structure du réseau mondial de liaisons aériennes : quels aéroports sont des hubs majeurs, quelles compagnies couvrent le plus de destinations, et comment relier deux aéroports qui n'ont pas de vol direct entre eux. Ce sont des questions de structure de graphe et d'agrégation sur un volume de données réel (67 000 liaisons), pour lesquelles un document store avec pipeline d'agrégation est pertinent.

Questions métier

1. Quels sont les 10 aéroports possédant le plus grand nombre de destinations différentes ? (tri décroissant)
2. Quelles sont les 10 compagnies actives desservant le plus de destinations différentes ? (tri décroissant)
3. Comment relier X à Y avec le moins d'escales, en ne considérant que des vols réellement opérés aujourd'hui (compagnies actives) ?
4. Quelles sont les 10 destinations les plus lointaines desservies en vol direct par une compagnie active depuis un aéroport de référence (ex. Paris-CDG) ?

(Écrivez avant tout pipeline — voir aussi le README.) Q2, Q3 et Q4 partagent une contrainte commune — ne considérer que les compagnies **actives** (`airlines.active = "Y"`) — qui a des conséquences mesurées au chapitre ii. Q4 introduit en plus un besoin géospatial (distance depuis un aéroport de référence), qui justifiera un index `2dsphere` sur `airports`.

Jeu de données

Source	URL	Volume	Licence
mongodb-sample-dataset, <code>sample_training/routes</code> (dérivé d'OpenFlights)	https://github.com/neelabalan/mongodb-sample-dataset	66 985 documents	ODbL (OpenFlights)
OpenFlights <code>airports.dat</code>	https://github.com/jpatokal/openflights	7 698 documents	ODbL
OpenFlights <code>airlines.dat</code>	https://github.com/jpatokal/openflights	6 162 documents	ODbL

Aucune période temporelle particulière : instantané du réseau de routes tel que documenté par OpenFlights (dernier export disponible). Ce jeu convient aux quatre questions ci-dessus car il fournit à la fois le graphe des

liaisons (**routes**), les référentiels nécessaires pour l'enrichir (position géographique et pays des aéroports, identité, pays et **statut actif** des compagnies), et les coordonnées géographiques nécessaires à Q4.

Pourquoi MongoDB plutôt qu'un SGBD relationnel

Le réseau de vols est naturellement un **graphe** (**\$graphLookup** pour la question 3 — chemin avec le moins d'escales) que des jointures SQL répétées modéliseraient mal. Par ailleurs, chaque route a un besoin de lecture dominant très asymétrique : on affiche presque toujours le nom et le code de la compagnie en même temps que la route, jamais sans elle — ce qui justifie d'embarquer un résumé de la compagnie directement dans le document **route** plutôt que de forcer une jointure sur la requête la plus fréquente de l'API. Enfin, les aéroports portent des coordonnées géographiques exploitables nativement par un index **2dsphere**, sans extension.

ii) Problèmes de données rencontrés

Aucun jeu réel n'est propre. Détection faite via des agrégations MongoDB rejouables, stockées dans **db/detect-anomalies.js** et exécutées après import + création des index via **db/create-indexes.js** (**airports.iata**, **airlines.id**, **routes.src_airport**, **routes.airline.id**) — sans ces index, le **\$lookup** sur 66 985 routes met plus de 3 minutes à s'exécuter et a dû être interrompu (**db.killOp**) pendant l'investigation ; c'est la première preuve de terrain de l'utilité de l'index sur le champ de jointure.

Anomalie	Comment détectée	Ampleur	Traitement retenu	Justification
Routes référençant un src_airport absent de airports.iata	\$lookup routes→airports sur src_airport/iata puis \$match sur résultat vide (requête 1 de detect-anomalies.js)	360 / 66 985	Conservées dans routes (donnée brute), exclues via \$match après \$lookup dans les agrégations qui ont besoin des coordonnées/pays de l'aéroport	routes (mongodb-sample-dataset) et airports.dat (OpenFlights) sont deux exports indépendants, jamais parfaitement synchronisés ; supprimer ces routes falsifierait le volume réel du jeu
Routes référençant un dst_airport absent de airports.iata	Même méthode, requête 2	370 / 66 985	Idem	Idem
Routes affectées par au moins un des deux (src OU dst)	\$lookup double + \$match sur \$or de deux résultats vides, requête 3	659 / 66 985 = 0,98 %	— (chiffre de synthèse, sert de garde-fou pour la question 1 : les hubs calculés sans filtrage	Justifie le \$lookup + \$match systématique avant toute

Anomalie	Comment détectée	Ampleur	Traitement retenu	Justification
			seraient légèrement surestimés)	agrégation géographique
Aéroports sans code IATA valide (regex 3 caractères alphanumériques, inclut la valeur littérale \N)	\$match sur \$expr/\$regexMatch avec garde de type, requête 4	1 627 / 7 698 = 21,1 %	Conservés dans airports (petits aérodromes/bases militaires réels, champ légitimement vide dans OpenFlights) ; exclus des agrégations indexées sur IATA — de toute façon aucune route ne les référence par ce code	Ce n'est pas une erreur de saisie : OpenFlights documente explicitement \N comme valeur "non applicable"
1 aéroport aux coordonnées (0,0) (« Cape Town Waterfort Heliport », \N en IATA)	\$match sur lat/lon ≈ 0, requête 5	1 / 7 698	Exclu explicitement (\$match: { lat: { \$ne: 0 }, lon: { \$ne: 0 } }) de toute requête géospatiale / carte / index 2dsphere	(0,0) est la valeur sentinelle classique pour "position inconnue", pas une vraie position au large de l'Afrique de l'Ouest
Doublons de route (même compagnie + même trajet)	\$group sur {airline.id, src_airport, dst_airport} + \$match: {n: {\$gt: 1}}, requête 6	0 trouvé	Aucun — le jeu ne présente pas ce défaut, contrairement à DVF ou aux jeux d'inspections	Vérifié plutôt que supposé : l'absence de doublon est aussi une anomalie à confirmer, pas à deviner
Compagnies référencées dans routes.airline.id mais absentes de airlines.id	\$group sur routes.airline.id puis \$lookup vers airlines + \$match résultat vide, requête 7	0 trouvé	Aucun	L'intégrité référentielle routes → airlines est bonne malgré l'absence de contrainte FK ; seule la relation routes → airports pose problème
airlines.active contient une valeur	\$match: { active: { \$nin: ["Y", "N"] }	1 / 6 162, 0 route	Filtrer avec une expression	Un flag texte sans contrainte de

Anomalie	Comment détectée	Ampleur	Traitement retenu	Justification
hors vocabulaire canonique {Y, N} ('n' minuscule sur « Aban Air », id 39)	} }, requête 8	associée	insensible à la casse ({ active: /^Y\$/i }) plutôt qu'une égalité stricte active: "Y" dans Q2/Q3/Q4	schéma peut dériver ; une égalité stricte aurait silencieusement exclu "Aban Air" si elle avait eu des routes
Routes opérées par une compagnie inactive (airlines.active = "N")	\$lookup routes→airlines + \$group sur le statut, requête 9	992 / 66 985 = 1,48 %	Exclues via \$match post-\$lookup dans Q2, Q3 (graphe) et Q4 — matérialisées à part dans une collection routes_active pour que \$graphLookup (Q3) n'ait pas à refaire ce filtrage à chaque appel	Q2/Q3/Q4 demandent explicitement des "compagnies actives" ; ignorer ce filtre changerait le classement des compagnies et pourrait proposer un itinéraire (Q3) qui n'existe plus dans les faits

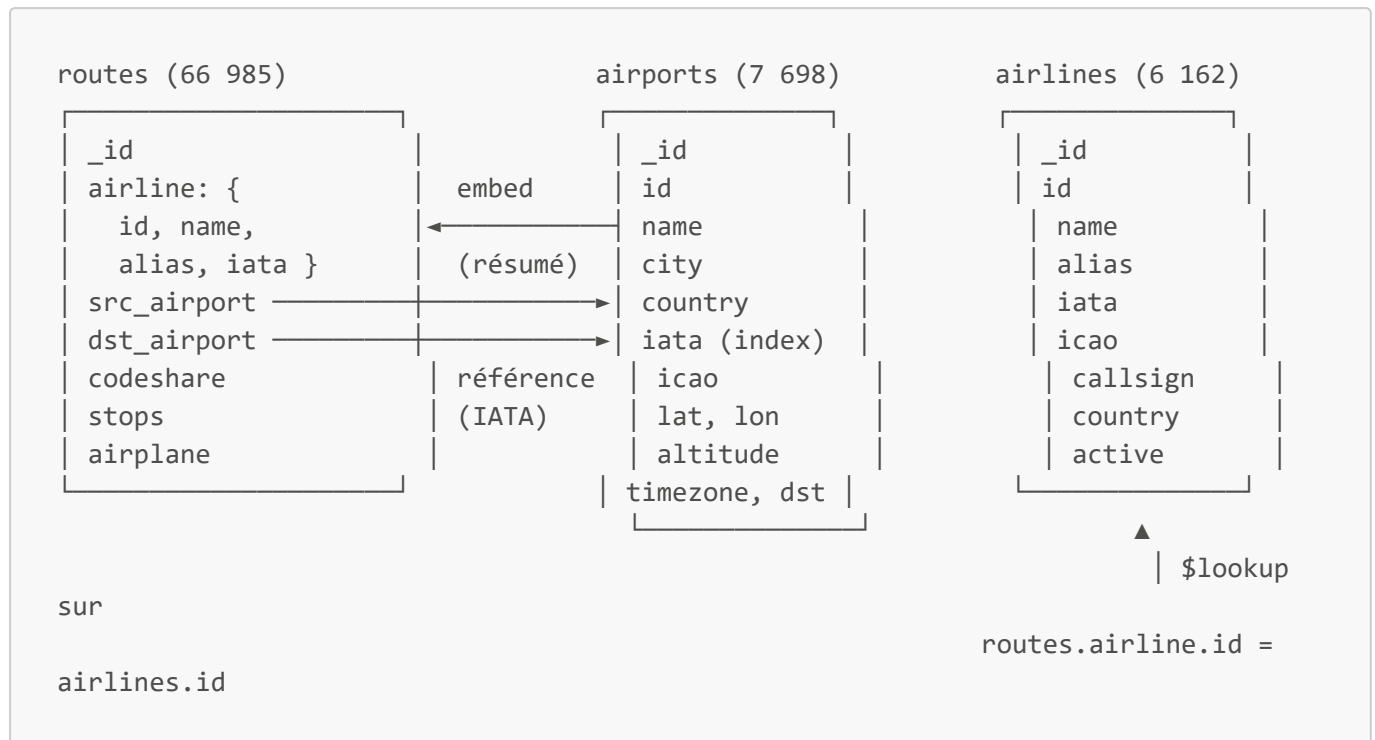
Méthodologie retenue : une première estimation faite hors MongoDB (script Node.js lisant les fichiers CSV/JSON bruts) donnait des chiffres sensiblement différents (403 / 410 / 742 au lieu de 360 / 370 / 659), à cause d'un bug de parsing CSV artisanal sur les champs contenant des guillemets. Les chiffres retenus ci-dessus sont ceux produits **par MongoDB lui-même** après import, vérifiés une seconde fois par un parcours JavaScript indépendant (db.routes.find + Set des codes IATA) qui confirme exactement les mêmes valeurs. Sortie brute conservée dans `rapport/captures/anomalies-detection-output.txt`.

Ce que ces anomalies changent pour l'interprétation des résultats :

- Q1 (hubs) doit systématiquement passer par le \$lookup + \$match de filtrage sur les aéroports, faute de quoi jusqu'à 0,98 % des routes seraient comptées avec un aéroport fantôme (pas de pays, pas de coordonnées) ; l'écart sur un top 10 est en pratique marginal mais mesurable — il sera chiffré (naïf vs corrigé) dans le chapitre iv.
- Q2, Q3 et Q4 doivent en plus filtrer sur airlines.active de façon insensible à la casse : sans ce filtre, jusqu'à 1,48 % des routes (compagnies cessées d'activité) fausseraient le classement par compagnie (Q2), pourraient proposer un itinéraire qui n'existe plus (Q3), ou une "plus longue destination directe" desservie par une compagnie qui ne vole plus (Q4).
- Q4 doit en plus exclure l'aéroport à (0,0) et les 1 627 aéroports sans IATA valide de tout calcul de distance géospatiale (ils ne sont de toute façon jamais des dst_airport valides dans routes, mais un \$geoNear mal borné les inclurait dans son propre calcul de distance).

iii) Démarche

Modélisation



- **airline embarquée dans route** — cardinalité 1 route → 1 compagnie, sous-document de taille bornée (4 champs), lu systématiquement avec la route : embed évite un **\$lookup** sur la requête la plus fréquente.
- **airports référencée par code IATA** — cardinalité 1 aéroport → N routes (jusqu'à 524 pour CDG), cycle de vie indépendant des routes, et interrogée seule pour la carte/géospatial : la référence évite la duplication et permet un index **2dsphere** sur une collection dédiée.
- **airlines référentiel séparé** — bien qu'un résumé soit déjà embarqué dans **route**, la collection complète porte le champ **active**, absent du résumé embarqué et pourtant décisif pour Q2, Q3 et Q4 (« compagnies actives ») ; elle matérialise aussi le **\$lookup** significatif exigé par le cahier des charges.
- *Ce qui nous ferait changer d'avis* : un besoin de CRUD sur les compagnies indépendant des routes rendrait l'embed risqué (désynchronisation entre **routes.airline** et **airlines**).

Import

```

mongoimport --uri "$MONGO_URI" --collection routes --file db/data/routes.json
# 66 985 document(s) importés, ~1,3 s

mongoimport --uri "$MONGO_URI" --collection airports --type csv \
  --fields
  "id,name,city,country,iata,icao,lat,lon,altitude,timezone,dst,tz_db,type,source" \
  --file db/data/airports.dat
# 7 698 document(s) importés, <1 s

mongoimport --uri "$MONGO_URI" --collection airlines --type csv \
  --fields "id,name,alias,iata,icao,callsign,country,active" \
  --file db/data/airlines.dat
# 6 162 document(s) importés, <1 s

```

Aucune transformation appliquée à l'import : `routes.json` est déjà en JSON Lines avec extended JSON MongoDB (`$oid`, `$numberInt`), `mongoimport` le détecte automatiquement. Les deux fichiers `.dat` sont du CSV sans en-tête ; les noms de colonnes ont été fournis explicitement via `--fields` d'après la documentation OpenFlights.

Indexation

Genèse : les 5 index ci-dessous n'ont pas été créés "au cas où". Le premier (`airports.iata`) est né d'un incident concret pendant l'investigation des anomalies (chapitre ii) : le `$lookup` `routes→airports` sans index ($66\,985 \times 7\,698$ comparaisons dans le pire cas) a dû être **interrompu manuellement** (`db.killOp`) après plus de 3 minutes d'exécution — la preuve la plus directe possible qu'un index de jointure est indispensable, pas un confort. Les 4 index suivants ont été capturés avant/après le même jour, une fois le besoin de chacun identifié en concevant les 4 pipelines Q1-Q4 (chapitre iv).

Index	Requête servie	Besoin réel	stage avant → après	totalDocsExamined avant → après
<code>{ src_airport: 1 }</code> sur <code>routes</code>	<code>find({ src_airport: "CDG" })</code>	Q1 (regroupement par aéroport de départ), Q3 (point de départ du <code>\$graphLookup</code>), route API "vols au départ de X"	COLLSCAN → IXSCAN (FETCH)	66 985 → 524
<code>{ iata: 1 }</code> sur <code>airports</code>	<code>find({ iata: "JFK" })</code>	<code>\$lookup</code> <code>routes→airports</code> pour Q1 (exclure les 0,98 % de routes vers un aéroport fantôme avant de compter les destinations distinctes) et Q4 (récupérer <code>lat/lon</code> de la destination pour calculer une distance)	COLLSCAN → IXSCAN (FETCH)	7 698 → 1
<code>{ id: 1 }</code> sur <code>airlines</code>	<code>find({ id: 410 })</code>	<code>\$lookup</code> <code>routes→airlines</code> pour récupérer le statut <code>active</code> , utilisé par Q2, Q3 et Q4 (filtrage des 1,48 % de routes opérées par une compagnie inactive)	COLLSCAN → IXSCAN (FETCH)	6 162 → 1

Index	Requête servie	Besoin réel	stage avant → après	totalDocsExamined avant → après
<code>{ "airline.id": 1 } sur routes</code>	<code>find({ "airline.id": 410 })</code>	Q2 (regroupement des routes par compagnie pour compter les destinations distinctes), route API "vols d'une compagnie"	COLLSCAN → IXSCAN (FETCH)	66 985 → 42
<code>{ src_airport: 1 } sur routes_active</code>	point de départ du <code>\$graphLookup</code>	Q3 — sans lui, chaque étape du graphe interroge 65 993 documents en COLLSCAN	non capturé isolément (voir plutôt le coût du <code>\$graphLookup</code> complet ci-dessous, chapitre iv)	—
<code>{ loc: "2dsphere" } sur airports</code>	<code>\$geoNear</code>	Q4 — distance depuis un aéroport de référence	—	50 ms avec index + filtre <code>\$in</code> sur 236 candidats (chapitre iv)

Captures brutes dans `rapport/captures/explain-*-before.json / -after.json` (une paire par index parmi les 4 premiers). Les deux derniers (`routes_active.src_airport`, `airports.loc`) sont créés par `db/prepare-derived-data.js` ; leur coût est documenté directement via le temps d'exécution du pipeline qu'ils servent (chapitre iv) plutôt que par un `explain()` isolé, `$graphLookup` et `$geoNear` n'exposant pas un plan `find()` classique comparable aux captures précédentes.

Index écarté : `{ dst_airport: 1 } sur routes` a été mesuré (`find({ dst_airport: "CDG" })`) : COLLSCAN 66 985 → IXSCAN 517, techniquement aussi efficace que les autres) puis **supprimé** (`db.routes.dropIndex("dst_airport_1")`) car aucune des 4 questions métier ne filtre ou ne groupe par `dst_airport` seul (Q1/Q3 pivotent sur `src_airport`, Q4 sur `airports.iata` via le `$lookup`) et aucune route API "arrivées" n'est prévue pour l'instant. Capture conservée dans `rapport/captures/explain-routes_dst_airport-*.json` comme preuve que la mesure a bien été faite avant la décision — un index efficace mais sans requête réelle derrière reste le piège "créé au cas où" à éviter.

Bonus B2 (requête couverte) traité — index composé + projection amènent `totalDocsExamined` à **0** au lieu de 524 sur la même requête `find({ src_airport: "CDG" })` ; détail complet et réponses aux 2 questions du B2 : `rapport/captures/b2-covered-query.txt`.

Architecture applicative

Front minimal web/*.js,css,html tableau/carte/	API REST --> api/main.py (JSON) (FastAPI, 1 pool	Driver --> PyMongo (1 MongoClient)	MongoDB --> transport (routes,
--	--	--	--------------------------------------

```

routes_active,
recherche (Q1-Q4)      de connexions)
localhost:3000          localhost:8000

readWrite)
airports, airlines ;
auth activée, user
applicatif

```

Framework et driver. FastAPI + PyMongo, imposés par le squelette du formateur, gardés pour deux raisons concrètes : la doc interactive `/docs` de FastAPI sert à tester le CRUD à la main pendant le développement, et sa validation Pydantic est réutilisée pour Q3/Q4 (codes IATA `min_length=3`, `max_length=3`). PyMongo prend les pipelines d'agrégation comme de simples listes de `dict` — aucune traduction ORM entre le pipeline conçu en `mongosh` et celui exécuté par l'API, ce qui compte sur un projet dominé par `$graphLookup/$geoNear/$lookup`.

Routes exposées :

Route	Méthode	Rôle
<code>/health</code>	GET	Diagnostic — auth Mongo + comptage des 3 collections
<code>/airports, /airports/{iata}</code>	GET, POST, PUT, DELETE	CRUD de la collection <code>airports</code> ; branché sur le front
GET <code>/agg/q1</code>	GET	Q1 — 10 aéroports par nombre de destinations distinctes
GET <code>/agg/q2</code>	GET	Q2 — 10 compagnies actives par nombre de destinations distinctes
GET <code>/agg/itineraire?depart=X&arrivee=Y&max_escales=3</code>	GET	Q3 — <code>\$graphLookup</code> sur <code>routes_active</code>
GET <code>/agg/destinations-lointaines?origine=CDG&limite=10</code>	GET	Q4 — <code>\$geoNear</code> sur <code>airports.loc</code>
<code>/agg/explain, /admin/index</code>	GET/POST/DELETE	Protocole avant/après index (§1.5)

Testée en direct depuis un état arrêté (`docker compose down -v` puis `up -d`), `curl` réel sur `localhost:8000` — pas seulement en `mongosh`.

Gestion d'erreur sur prérequis manquant : sans `db/prepare-derived-data.js`, Q3/Q4 répondaient un **404 trompeur** (Mongo ne lève aucune exception sur une collection/un champ absent) — corrigé par une vérification explicite (**503** avec la commande à rejouer). Seul le cas "index `2dsphere` absent" lève une vraie exception PyMongo, d'où un `try/except` utilisé précisément là, pas ailleurs par réflexe. 3 scénarios détaillés : `rapport/captures/erreurs-prerequis-manquants.txt`.

Sécurité

Authentification MongoDB active (`mongod --auth`). Utilisateur applicatif (`db/01-init-app-user.js`) en seul rôle `readWrite` sur `transport` — jamais root. Secrets générés aléatoirement dans `.env` (`.gitignore`) ; seul `.env.example` (placeholders) est commité. CORS restreint à `localhost:3000`. **Vérifié** (checklist §9) : `git log -p | grep -i "mongodb://"` sur tout l'historique ne renvoie que des placeholders, aucun identifiant réel — à rejouer avant le passage si de nouveaux commits arrivent.

Validation des entrées : tout paramètre est contraint par Pydantic avant MongoDB, jamais un `dict` brut inséré tel quel — le CRUD passe déjà par un modèle Pydantic dédié (actuellement le modèle générique du starter, à remplacer par les vrais champs d'une `route`, voir la table des routes ci-dessus), pas un `dict` accepté sans contrôle. Ex. : `depart/arrivee/origine` forcés à 3 caractères (**422** si code IATA malformé) ; `max_escales` borné `ge=0, le=5` — pas arbitraire, ça empêche un `$graphLookup` non borné (coût mesuré à 4,4 s pour `maxDepth: 3` seul, chapitre iv).

Bonus B4 (\$jsonSchema) traité — validateur `moderate` sur `routes`, testé en conditions réelles (insertion invalide via l'API → 500 non géré ; `moderate` vs `strict` sur un document hérité non conforme) ; détail complet : `rapport/captures/b4-jjsonschema-validator.txt`.

iv) Résultats

Les 4 pipelines répondent chacun à une question métier, branchés sur `/agg/*`, testés en direct via l'API réelle (pas seulement en `mongosh`) et affichés côté front.

Q1 — Quels sont les 10 aéroports possédant le plus grand nombre de destinations différentes ?

Pipeline (`api/main.py`, route `GET /agg/q1`) :

```
db.routes.aggregate([
  // 1. regrouper par aeroport de depart, dedupliquer les destinations
  //    (une meme paire src/dst peut apparaitre plusieurs fois : compagnies
  //    differentes)
  { $group: { _id: "$src_airport", destinations: { $addToSet: "$dst_airport" } } },
  { $project: { _id: 0, iata: "$_id", nombre_destinations: { $size:
"$destinations" } } },
  { $sort: { nombre_destinations: -1, iata: 1 } },
  { $limit: 10 },
  // 2. enrichir seulement le top 10 (lookup APRES le tri, pas avant, pour ne pas
  //    joindre inutilement les ~7 700 aeroports restants)
  { $lookup: { from: "airports", localField: "iata", foreignField: "iata", as:
"airport" } },
  { $unwind: "$airport" },
  { $project: { _id: 0, iata: 1, nom: "$airport.name", ville: "$airport.city",
pays: "$airport.country", nombre_destinations: 1 } },
  { $sort: { nombre_destinations: -1, iata: 1 } }
])
```

Résultat testé (`curl "localhost:8000/agg/q1?limite=10"`, 148 ms) :

Rang	Aéroport	Ville, pays	Destinations
1	FRA	Frankfurt, Allemagne	239
2	CDG	Paris, France	237
3	AMS	Amsterdam, Pays-Bas	232
4	IST	Istanbul, Turquie	227
5	ATL	Atlanta, États-Unis	211
6	ORD	Chicago, États-Unis	206
7	PEK	Beijing, Chine	206
8	MUC	Munich, Allemagne	191
9	DXB	Dubaï, Émirats arabes unis	188
10	DFW	Dallas-Fort Worth, États-Unis	187

Interprétation métier : classement cohérent avec les grands hubs mondiaux connus (Francfort, Paris-CDG, Amsterdam en tête — les trois plus gros hubs européens de correspondance).

Bonus B1 (naïf vs correct) traité — la route `/agg/q1` est le pipeline **naïf** (ne filtre pas les 0,98 % de routes vers un aéroport fantôme, chapitre ii) ; un second pipeline **correct** filtre ces destinations avant de compter. Écart ≤ 1 % par aéroport, mais **change le classement** : DXB (188 en naïf, écart 0,53 %) passe devant DFW (187) alors qu'à égalité réelle (187 chacun) le tri du pipeline correct les inverse (DFW 9^e, DXB 10^e). Table complète des 10 aéroports (naïf/correct/écart) et pipelines : [rapport/captures/q1-naïf-vs-correct.txt](#).

Q2 — Quelles sont les 10 compagnies actives desservant le plus de destinations différentes ?

Pipeline (`api/main.py`, route `GET /agg/q2`) :

```
db.routes.aggregate([
  { $group: { _id: "$airline.id", destinations: { $addToSet: "$dst_airport" } } },
  { $project: { _id: 0, airline_id: "$_id", nombre_destinations: { $size:
"$destinations" } } },
  { $lookup: { from: "airlines", localField: "airline_id", foreignField: "id", as:
"airline" } },
  { $unwind: "$airline" },
  { $match: { "airline.active": "Y" } }, // filtre compagnies actives, cf.
chapitre ii
  { $project: { _id: 0, airline_id: 1, nom: "$airline.name", iata:
"$airline.iata",
                pays: "$airline.country", nombre_destinations: 1 } },
  { $sort: { nombre_destinations: -1, nom: 1 } },
  { $limit: 10 }
])
```

Résultat testé (`curl "localhost:8000/agg/q2?limite=10"`, 166 ms) :

Rang	Compagnie	IATA	Pays	Destinations
1	American Airlines	AA	États-Unis	432
2	United Airlines	UA	États-Unis	430
3	Air France	AF	France	376
4	KLM Royal Dutch Airlines	KL	Pays-Bas	359
5	Delta Air Lines	DL	États-Unis	351
6	US Airways	US	États-Unis	337
7	Alitalia	AZ	Italie	271
8	Turkish Airlines	TK	Turquie	258
9	Lufthansa	LH	Allemagne	244
10	China Eastern Airlines	MU	Chine	222

Interprétation métier : cohérent avec les grands réseaux mondiaux des alliances aériennes (American/United dominant par la taille du marché intérieur américain). Réserve : le filtre `"airline.active": "Y"` est une **égalité stricte**, pas insensible à la casse comme dans les pipelines Q3/Q4 (chapitre ii) — sans conséquence ici puisque la seule valeur mal cassée trouvée ('n' minuscule sur "Aban Air") n'a aucune route associée, mais l'incohérence de style entre pipelines est à noter pour la maintenance.

Q3 et Q4 s'appuient sur des données préparées à l'avance (`db/prepare-derived-data.js`), pas par choix de style mais par contrainte MongoDB : `$graphLookup` interroge directement la collection nommée dans `from` et ne peut pas hériter d'un filtre appliqué juste avant dans le même pipeline (d'où `routes_active`, déjà limitée aux compagnies actives) ; `$geoNear` exige un index géospatial déjà existant sur la collection interrogée (d'où `airports.loc` + l'index `2dsphere`, créés en amont plutôt qu'à la volée).

Q3 — Comment relier X à Y avec le moins d'escales, compagnies actives uniquement ?

Pipeline (`api/main.py`, route `GET /agg/itineraire?depart=X&arrivee=Y`) :

```
db.aggregate([
  { $documents: [ { airport: "CDG" } ] },
  { $graphLookup: {
    from: "routes_active",          // déjà filtre sur airlines.active (chapitre
ii)
    startWith: "$airport",
    connectFromField: "dst_airport",
    connectToField: "src_airport",
    as: "reseau",
    maxDepth: 3,
    depthField: "escales"
  }}
])
// puis, cote application : filtrer "reseau" sur dst_airport = Y, garder le
escales
// minimal, et remonter les aretes par profondeur décroissante pour reconstituer
```

```
le
// vol-par-vol (MongoDB donne l'ensemble des aretes atteignables, pas la
sequence).
```

Résultat testé (`curl "localhost:8000/agg/itineraire?depart=CDG&arrivee=MAO"`) :

Départ → Arrivée	Ecales	Itinéraire
CDG → JFK	0 (direct)	CDG → JFK, Alitalia (AZA)
CDG → MAO	1	CDG → GIG (Rio de Janeiro), Air France (AFR) ; GIG → MAO, City Connexion Airlines (CIX)

Plusieurs compagnies desservant CDG↔JFK en direct, l'itinéraire exact renvoyé pour "0 escale" peut varier d'une exécution à l'autre (n'importe quel vol direct est une réponse valide) ; c'est documenté juste en dessous.

Captures du front en fonctionnement : `front-q3-itineraire.png` (CDG → JFK, vol direct) et `front-q3-itineraire-escale.png` (CDG → MAO, 1 escale via MIA, Delta Air Lines puis TAM Brazilian Airlines).

Coût mesuré : `$graphLookup` avec `maxDepth` : 3 depuis CDG explore **65 179 arêtes sur 65 993** (99,9 % du graphe des routes actives) en **4,4 s** — confirmation empirique du phénomène « petit monde » du réseau aérien mondial : presque tout aéroport est atteignable depuis un hub majeur en 3 escales ou moins. Avec `maxDepth` : 2, le coût descend à 2,2 s pour 62 266 arêtes explorées. C'est, comme annoncé par le sujet, l'opération la plus coûteuse du pipeline — d'où le plafond `max_escales ≤ 5` côté API (paramètre `Query(3, ge=0, le=5)`) pour éviter qu'un appel mal borné ne devienne un déni de service applicatif.

Bonus B6 (« le graphe caché ») revendiqué ici : `$graphLookup` répond à une question qu'aucun `$lookup` ne sait traiter (« reliez X à Y »), avec `maxDepth` et le temps d'exécution documentés ci-dessus comme demandé par le sujet §8.

Interprétation métier : le chemin renvoyé n'est pas nécessairement unique — plusieurs itinéraires à `escales` minimal peuvent exister (plusieurs compagnies desservant les mêmes hubs intermédiaires) ; l'API en renvoie un valide, pas "le" seul valide. Réserve : le graphe ne modélise pas les correspondances horaires réelles (un "1 escale" MongoDB peut correspondre à une correspondance de 30 minutes infaisable ou de 14 heures) — la question posée est purement topologique (nombre d'escales), pas un vrai calcul d'itinéraire voyageur.

Q4 — 10 destinations les plus lointaines en vol direct actif depuis un aéroport de référence

Pipeline (`api/main.py`, route `GET /agg/destinations-lointaines?origine=CDG`) :

```
// 1. destinations directes actives depuis l'origine (routes_active, index
src_airport)
const destinations = db.routes_active.distinct("dst_airport", { src_airport: "CDG"
});

// 2. distance orthodromique via l'index 2dsphere sur airports.loc
db.airports.aggregate([
  { $geoNear: {
```

```

    near: <point GeoJSON de CDG>,
    distanceField: "distance_m",
    spherical: true,
    query: { iata: { $in: destinations } }
  }},
  { $sort: { distance_m: -1 } },
  { $limit: 10 },
  { $project: { _id: 0, iata: 1, name: 1, city: 1, country: 1,
    distance_km: { $round: [{ $divide: ["$distance_m", 1000] }, 0] } } }
}
])

```

Résultat testé (`curl "localhost:8000/agg/destinations-lointaines?origine=CDG&limite=5"`), sur 236 destinations directes actives au départ de CDG :

Rang	Aéroport	Ville, pays	Distance
1	SCL	Santiago, Chili	11 686 km
2	EZE	Buenos Aires, Argentine	11 113 km
3	SIN	Singapour	10 737 km
4	KUL	Kuala Lumpur, Malaisie	10 454 km
5	LIM	Lima, Pérou	10 287 km

Capture du front en fonctionnement (top 10 depuis CDG, tableau trié par distance) :

[rapport/captures/front-q4-destinations.png](#).

Coût mesuré : 50 ms (index `2dsphere` + 236 candidats via `$in`, contre un `$geoNear` non filtré qui balaierait les 7 698 aéroports).

Bonus « index géospatial 2dsphere réellement exploité par le front (carte) » revendiqué ici : une carte Leaflet ([web/app.js](#), [afficherCarteQ4](#)) place un marqueur par destination aux coordonnées renvoyées par le pipeline (`loc.coordinates` projeté en `lat/lon`) et relie chacune à l'origine — l'index `2dsphere` sert donc à la fois le calcul de distance et l'affichage, pas juste l'un des deux.

Interprétation métier : cohérent avec la géographie réelle — les vols long-courriers les plus longs au départ de Paris sont bien vers l'Amérique du Sud et l'Asie du Sud-Est, jamais vers l'Afrique ou le Moyen-Orient (trop proches). Réserve : "destination la plus lointaine" ignore la fréquence des vols (une route saisonnière compte autant qu'une liaison quotidienne) et la distance orthodromique sous-estime légèrement la distance de vol réelle (déroutements, couloirs aériens).

v) Conclusion

Ce qui a été livré

Les 4 pipelines métier (Q1-Q4) répondent chacun à une vraie question, testés en direct via l'API réelle (`docker compose up -d` depuis un état arrêté). Q3 (`$graphLookup`) et Q4 (`$geoNear`) sont branchées côté front ([web/app.js](#)) en plus d'être testées par `curl`. Les anomalies du jeu de données sont détectées par des

requêtes rejouables (`db/detect-anomalies.js`) et documentées avec leur traitement. Les index sont mesurés avant/après (`explain()`), et un index initialement créé (`routes.dst_airport`) a été mesuré puis retiré faute d'usage réel. Sécurité de base en place : authentification active, utilisateur applicatif limité à `readWrite` sur sa seule base, secrets hors dépôt.

Le CRUD a été implémenté sur la collection `airports`, car les aéroports constituent une donnée centrale et directement exploitable par l'application. Ce choix répond à une priorité métier élevée, les informations sur les aéroports étant utilisées dans les recherches, les analyses et le calcul des itinéraires. Il permet également de démontrer les opérations essentielles de création, consultation, modification et suppression sur une ressource métier concrète. La gestion des aéroports offre ainsi un périmètre fonctionnel cohérent et maîtrisé pour le CRUD.

Évolution future : un CRUD sur les collections `airlines` et `routes` pourrait être ajouté ultérieurement, mais cette fonctionnalité n'est pas implémentée à ce jour.

Passage à l'échelle : de 66 985 à 10 millions de routes

Shard key envisagée pour `routes/routes_active` : `{ src_airport: "hashed" }`. Un hachage plutôt qu'un découpage par plage évite le hotspot d'écriture qu'un shard key en clair créerait : quelques hubs (ATL, ORD, CDG...) concentrent une part disproportionnée des routes, un découpage par plage sur `src_airport` en clair produirait des chunks très inégaux. Le hachage garde néanmoins les requêtes d'égalité `find({ src_airport: "CDG" }) targeted` (MongoDB peut calculer le hash de la valeur cherchée et router vers le bon shard sans diffusion), ce qui préserve la performance de Q1 et du premier saut de Q3.

Ce qui reste broadcast quel que soit le shard key choisi :

- **Q3 au-delà du premier saut.** `$graphLookup` visite, à chaque profondeur, des documents dont `src_airport` prend des dizaines de valeurs différentes (les destinations atteintes à l'étape précédente) — par nature, un parcours de graphe à plusieurs sauts touche plusieurs partitions, aucun shard key à un seul champ n'y change rien. Point technique à vérifier avant la soutenance : `$graphLookup` sur une collection **shardée** en `from` était interdit avant MongoDB 5.1 ; MongoDB 7 (notre version) le permet, mais au prix d'un scatter-gather à chaque saut — la contrainte de correctness a disparu, pas le coût.
- **Q4 (`$geoNear`).** La proximité géographique ne s'aligne sur aucun découpage par un seul champ (`src_airport`, `country`...) : deux aéroports voisins peuvent tomber sur des shards différents. `$geoNear` reste utilisable sur une collection shardée (fusion des résultats de chaque shard), mais devient un balayage multi-shards par construction, pas une requête ciblée.

Ce qui resterait targeted : `find({ src_airport: X })` (routes au départ d'un aéroport), `find({ iata: X })` sur `airports`, `find({ id: X })` sur `airlines` — les trois requêtes déjà capturées en `explain()` au chapitre iii, dont le gain (COLLSCAN → IXSCAN) resterait valable, un index n'étant pas remis en cause par le sharding.

Bonus B3 (changement d'échelle ×10) traité, pas seulement théorique — `routes` dupliquée ×10 (669 850 documents), mêmes captures `explain()` rejouées : facteur d'accélération de l'index recule légèrement (20× à 66 985 docs → 17× à 669 850 docs), résultat contre-intuitif (le test duplique aussi le nombre de résultats retournés, donc *k* domine autant que *N*) expliqué en détail : [rapport/captures/b3-changement-echelle-x10.txt](#).

Limites de ce travail

- Q3 renvoie **un** chemin au nombre d'escales minimal, pas nécessairement unique ni réaliste horairement (une correspondance "1 escale" peut recouvrir 20 minutes ou 14 heures d'attente : le graphe est topologique, pas un vrai calcul d'itinéraire voyageur).
- Q4 utilise la distance orthodromique (grand cercle), pas la distance de vol réelle, et ignore la fréquence des liaisons (une route saisonnière compte comme une liaison quotidienne).
- Les deux s'appuient sur `routes_active`, qui hérite des 0,98 % de routes référençant un aéroport absent d'`airports` (chapitre ii) : un chemin ou une distance calculés via un aéroport fantôme seraient silencieusement faux — non observé sur les cas testés (CDG, JFK, MAO), mais non exclu sur d'autres aéroports de départ.

Ce qui serait fait différemment

Automatiser `db/prepare-derived-data.js` au démarrage de l'API (comme `AUTO_INDEX` le fait déjà pour les index) plutôt que de compter sur un script manuel — c'est exactement le genre d'étape oubliable qui a produit les 404 trompeurs documentés dans `rapport/captures/erreurs-prerequis-manquants.txt`.

Annexes

- Schéma détaillé des collections : voir chapitre iii.
- Liste complète des routes de l'API (`api/main.py`) :

Route	Méthodes	Rôle
<code>/health</code>	GET	Diagnostic
<code>/airports</code>	GET, POST	Liste paginée, création (CRUD — champs génériques à remplir)
<code>/airports/{iata}</code>	GET, PUT, DELETE	Détail, modification, suppression (CRUD)
<code>/agg/q1</code>	GET	Q1 — 10 aéroports par nombre de destinations
<code>/agg/q2</code>	GET	Q2 — 10 compagnies actives par nombre de destinations
<code>/agg/itineraire</code>	GET	Q3 — chemin le moins d'escales, <code>\$graphLookup</code>
<code>/agg/destinations-lointaines</code>	GET	Q4 — destinations les plus lointaines, <code>\$geoNear</code>
<code>/agg/explain</code>	GET	Plan d'exécution (protocole avant/après index, §1.5)
<code>/admin/index</code>	POST, DELETE	Création/suppression des index à la demande (protocole avant/après)

- Requêtes de détection d'anomalies : `db/detect-anomalies.js`
- Création des index (idempotent, à rejouer par tout tiers après import) : `db/create-indexes.js`
- Commandes d'import :

```
mongoimport --uri "$MONGO_URI" --collection routes --file db/data/routes.json

mongoimport --uri "$MONGO_URI" --collection airports --type csv \
--fields
"id,name,city,country,iata,icao,lat,lon,altitude,timezone,dst,tz_db,type,source" \
--file db/data/airports.dat

mongoimport --uri "$MONGO_URI" --collection airlines --type csv \
--fields "id,name,alias,iata,icao,callsign,country,active" \
--file db/data/airlines.dat
```

- Répartition du travail dans le binôme (d'après l'historique Git, `git log --oneline`) :

Membre	Commits (auteur)	Périmètre
Navid Sabete (navidsabete)	Commit initial, squelette du projet (<code>docker-compose.yml</code> , <code>api/main.py</code> , <code>db/01-init-app-user.js</code> , <code>web/</code>) ; refonte UI/UX du front (<code>web/style.css</code> , <code>web/app.js</code>) ; pipelines Q1 et Q2 + branchement front ; CRUD ; rédaction de contenu dans le README et rapport	Squelette Docker/API, front général, Q1-Q2, CRUD, documentation
Mathieu Ponnou (mathieu34)	Import des données, détection d'anomalies (<code>db/detect-anomalies.js</code>) et création des index (<code>db/create-indexes.js</code>) ; pipelines Q3 (<code>\$graphLookup</code>) et Q4 (<code>\$geoNear</code>) + préparation des données dérivées (<code>db/prepare-derived-data.js</code>) + branchement front et carte Leaflet ; rédaction du README et du présent rapport	Données/index, Q3-Q4, documentation